

Prompting, Reasoning, & Memory

Shreya Havaldar

Announcements

- HW 1 due today at 11:59 pm EST
 - Reminder: 3 late days per semester
- HW 2 will be released tomorrow

Today's Lecture

1. **Prompting**
2. Reasoning techniques
3. Memory management

Prompts!

Key framing:

Prompts are not questions; they are specifications

Prompts as specifications

Main components:

1. Goal (what success means)
2. Inputs (what is provided; what is missing)
3. Constraints (must/never; policy; style; length)
4. Output format (schema; sections; deterministic formatting)
5. Acceptance tests (2–5 examples of correct/incorrect outputs)

Prompt Engineering

Prompts always involve instructions and context passed to an LLM to achieve a desired task.

Prompt engineering is the practice of developing and optimizing these prompts.

Typical prompt layout

Use consistent blocks to reduce ambiguity and improve debugging:

Role/Task: “You are ... your task is ...”

Rules: bullet list, straightforward information

Context: any needed facts or data

Output format: exact schema + “no extra text”

Examples: correct and incorrect (if needed)

Design Tips

1. Start simple
 - a. Prompting is an iterative process!
 - b. Experiment, tweak the prompt, repeat

Three main things to remember:

specificity, simplicity, and conciseness

Design Tips

2. Break things down into subtasks
 - a. LLMs are (usually) bad at doing many complex things at once
 - b. Helps you keep your prompts simple and specific.
 - c. Iteratively break down components your LLM struggles with

Design Tips

3. Be clear about the task and expected output
 - a. The more descriptive the prompt is, the better the results
 - b. But... too many (unnecessary) details may degrade results
 - c. Examples are almost always beneficial

A bad prompt

“Explain the concept prompt engineering. Keep the explanation short, only a few sentences, and don't be too descriptive”

- Length of explanation ❌
- Style of explanation ❌

A better prompt

“Use 2-3 sentences to explain the concept of prompt engineering to a high school student.”

- Length of explanation 
- Style of explanation 

A bad prompt

“You are an agent that recommends movies to a customer. DO NOT ASK FOR INTERESTS. DO NOT ASK FOR PERSONAL INFORMATION.”

- Output constraints ✗
- Edge cases ✗

A better prompt

“You are an agent that recommends movies to a customer. The agent should recommend a movie from the top global trending movies. It should not ask users for their preferences or personal information. If the agent doesn't have a movie to recommend, it should respond “Sorry, no movie available””

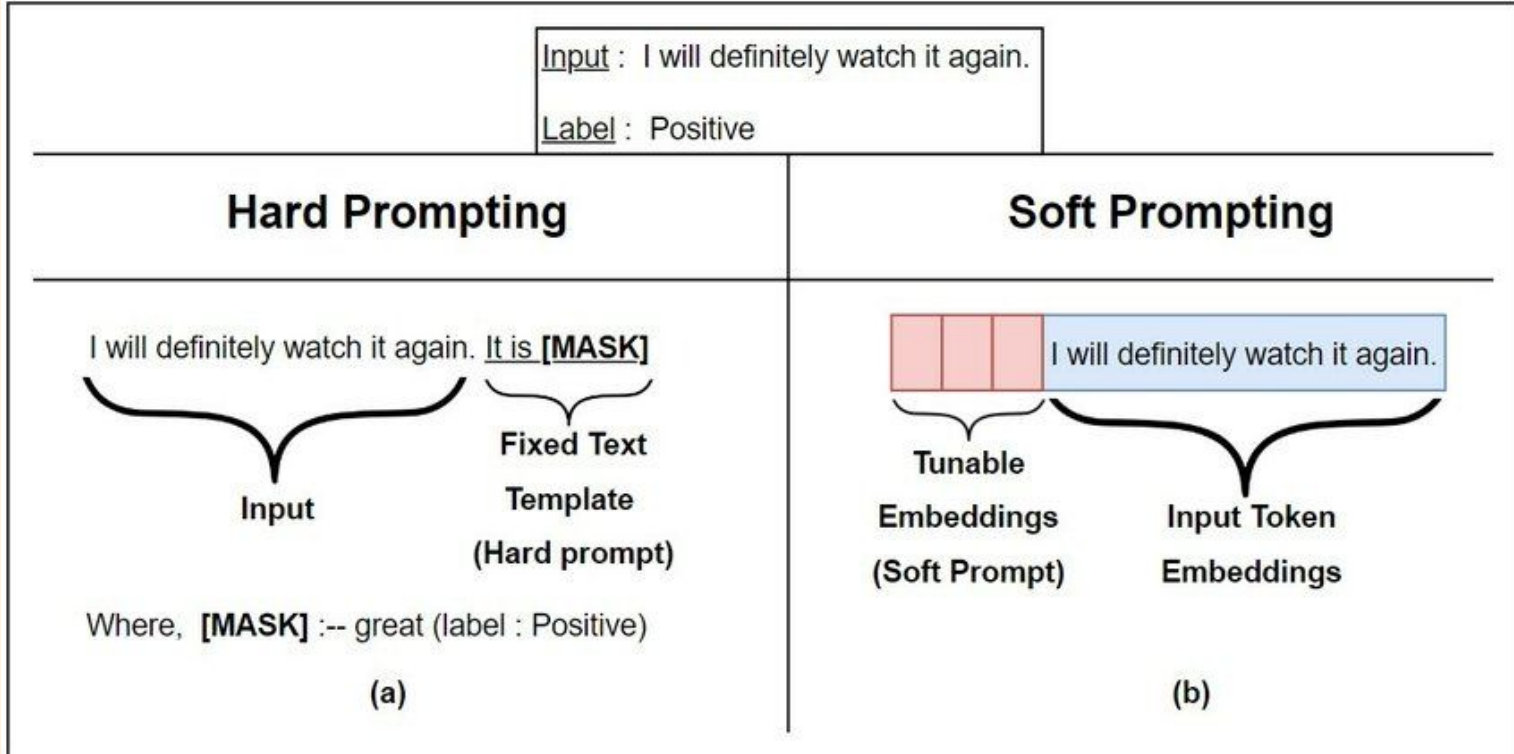
- Output constraints 
- Edge cases 

Soft Prompting

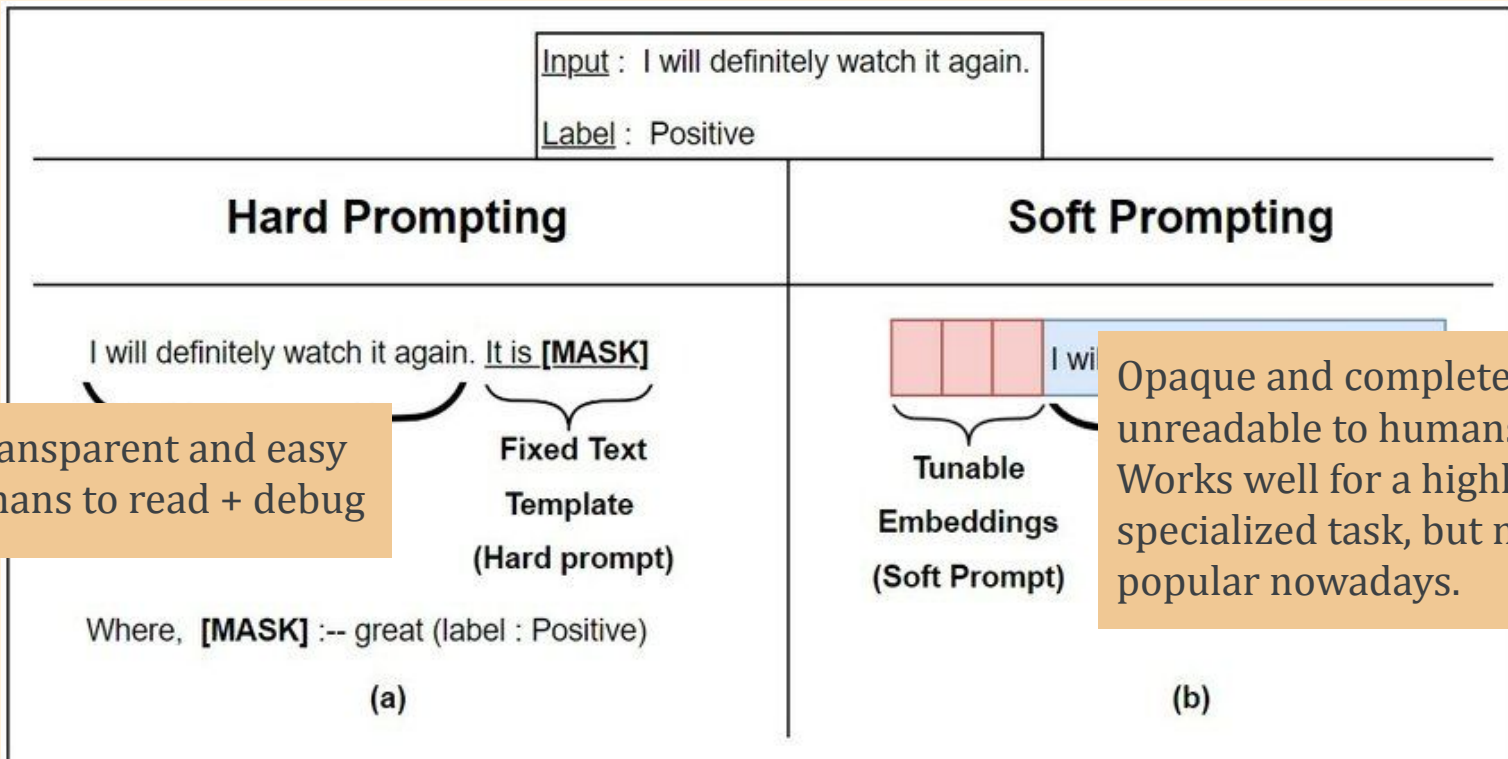
Soft prompting is a parameter-efficient technique that uses learned embeddings instead of human-readable text to guide LLMs

1. Appends trainable vectors (also called “virtual tokens”) directly to the input embedding sequence.
2. Keeps the core LLM weights completely frozen during the learning process.
3. Gradient Optimization: Updates only the small set of prompt parameters via backpropagation using a task-specific training dataset.

Soft Prompting



Soft Prompting



Fully transparent and easy for humans to read + debug

Opaque and completely unreadable to humans. Works well for a highly specialized task, but not popular nowadays.

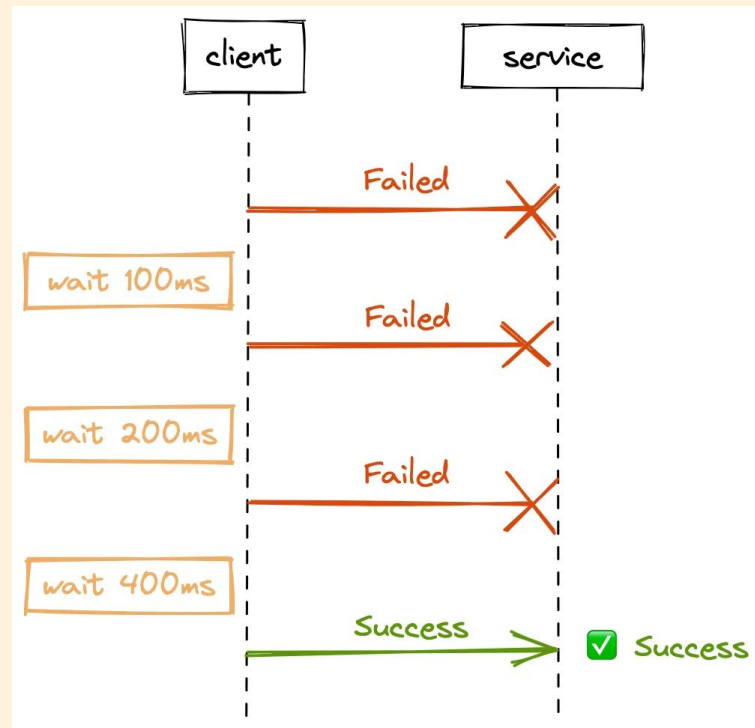
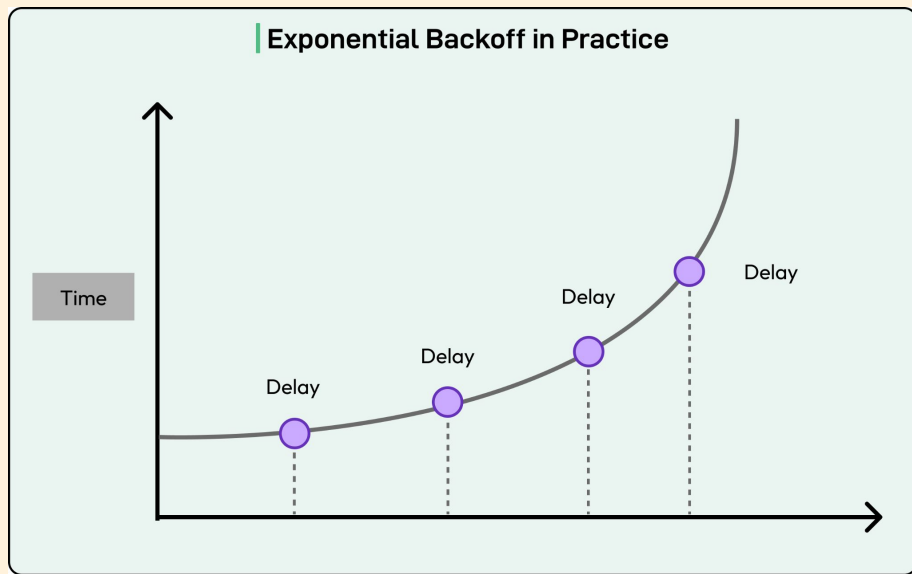
Prompting templates

Templates exist for most basic LLM tasks (like those here!)

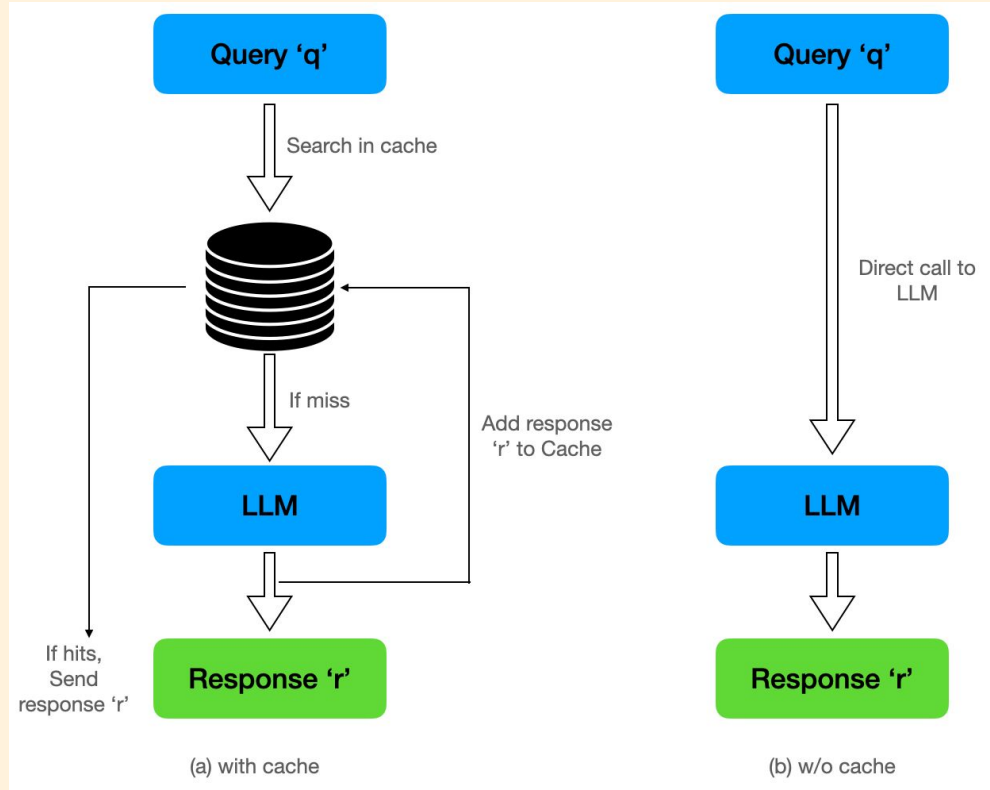
- Text Summarization
- Information Extraction
- Question Answering
- Text Classification
- Code Generation

Also useful: <https://chatgpt.com/use-cases/students>

Good practice: exponential backoff



Good practice: caching



Prompt caching
improves efficiency
and reduces costs for
repeat queries

```
def ask(prompt: str, temperature: float = 0.0) -> str:
```

```
    """
```

```
    Written for you. Sends one prompt to MODEL and returns the
    Every reply is saved to .autoprompting_cache.json, and an
    returns the saved reply, even at a temperature above 0. Do
    draw fresh replies. Waits and retries when the free tier
    hit.
```

```
    """
```

```
    global _client
```

```
    request = {"model": MODEL, "prompt": prompt, "temperature": temperature}
```

```
    key = hashlib.sha256(json.dumps(request, sort_keys=True).encode()).hexdigest()
```

```
    cache = json.loads(CACHE.read_text(encoding="utf-8")) if CACHE.exists() else {}
```

```
    if key in cache:
```

```
        return cache[key]
```

```
    if _client is None:
```

```
        ... < NORMAL API CALL CODE >
```

hashlib.sha256()
Converts any text or
file into a unique,
fixed 64-character
hexadecimal string

Good practice: API key safety

In terminal:

```
export GEMINI_API_KEY=<your_api_key>
```

In your code:

```
api_key=os.environ["GEMINI_API_KEY"]
```

- Storing API keys as environment variables is the industry standard for preventing accidental credential leaks
- Allows you to safely share or commit your codebase without exposing secret keys

Today's Lecture

1. Prompting
2. **Reasoning techniques**
3. Memory management

What is reasoning?

Reasoning in LLMs refers to the capability of models to solve complex, multi-step problems by breaking them down into intermediate, logical, and sequential steps

What is reasoning?

Reasoning in LLMs refers to the capability of models to solve complex, multi-step problems by breaking them down into intermediate, logical, and sequential steps

Reasoning in prompts: Taking any LLM and prompting it in a way that elicits reasoning capabilities

Reasoning models: LLMs that have been fine-tuned to internally reason before generating an output

When reasoning prompts help

Reasoning techniques typically help for:

- Multi-step arithmetic
- Multi-constraint decisions
- Compositional tasks
- Multi-hop QA.

They often don't help for:

- Missing knowledge
- Ambiguous tasks
- Tasks better solved by tools

Chain-of-Thought (CoT)

CoT improves multi-step reasoning by eliciting intermediate steps

→ “let’s think step-by-step”

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei Xuezhi Wang Dale Schuurmans Maarten Bosma

Brian Ichter Fei Xia Ed H. Chi Quoc V. Le Denny Zhou

Google Research, Brain Team
`{jasonwei,dennyzhou}@google.com`

Chain-of-Thought (CoT)

Types of CoT prompting:

Zero-shot: *task description only, with no examples given*

Few-shot: *task description with some amount of examples*

8-shot: *task description with exactly 8 examples*

When to use CoT: multi-step math, symbolic manipulation, compositional reasoning, planning subproblems.

Decomposition

Generic scaffold:

List the constraints

Break into 3–6 substeps

Solve each substep

Assemble final answer in the required format

“Least-to-most” style decomposition: Start with easy subquestions; use them as givens for the harder step.

Self-consistency

Uses sampling to choose the highest likelihood answer

1. Set the temperature to a higher value
 - a. → increases diversity in generations
2. Have the LLM produce N independent solutions
3. Choose the majority, or “most consistent” generation

Tree of Thoughts (ToT)

Generic scaffold:

```
Define "thought" unit (partial solution / subplan)
Generate k candidates
Score candidates (model-based evaluation, verifier)
Expand best nodes until depth/timeout
Return best leaf + justification
```

Generalizes CoT into a search over “thoughts” (branches), enabling exploration and lookahead, improving problems requiring search.

Program-Aided Language (PAL)

- Offloads execution correctness to a runtime (e.g., Python)
- Reduces arithmetic or logical slip-ups in final computation.
- Good for problems needing mathematical reasoning

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 tennis balls.

```
tennis_balls = 5
```

2 cans of 3 tennis balls each is

```
bought_balls = 2 * 3
```

tennis balls. The answer is

```
answer = tennis_balls + bought_balls
```

Reflexion

Reflexion converts feedback into self-reflections stored in memory.

These reflections are re-used in subsequent trials.

→ Improving performance without fine-tuning.

Generic scaffold:

Attempt → Evaluate → Write reflection → Next attempt
conditioned on reflection

Self-verification

The LLM has to “verify” it’s answer before continuing

- **Self-consistency:** Majority vote for single-answer tasks.
- **Checker model:** critique + conditioned regeneration.
- **Tool-based verification:** compute with interpreter; retrieve citations; cross-check constraints.
- **Invariant checks:** schema validity (e.g. correct json), policy constraints, numeric sanity checks

Reasoning failure modes

- **Constraint drop:** the LLM ignores a must/never rule
- **Premature closure:** answers before considering all constraints
- **Spurious precision:** invents missing values
- **Non-robust in long context:** fails when the relevant fact is not near the beginning/end

** Research shows that LLM performance can degrade when relevant information is positioned in the middle of a long input prompt, even for long-context models.*

Reasoning Models

Prompt engineering for complex reasoning problems is a challenging task.
This motivated reasoning models!

**Reasoning models generate internal chains of intermediate steps,
then select and refine a final answer.**

Recap: standard LLM inference

During generation, an LLM:

1. Tokenizes the input
2. Runs a forward pass
3. Gets a set of logits for every token in its vocabulary
4. Converts these logits to a probability distribution over tokens
5. Selects the output token using this distribution
6. Appends the output token to the generation; repeats

Reasoning LLM inference

Same as standard inference, but...

- The LLM first generates “**thinking tokens**”
 - Structured intermediate reasoning traces
 - Thinking step-by-step, decomposition, verification, etc.
- These “thinking tokens” are not part of the final answer

Can you design a lecture on safety mechanisms for Agentic AI?

Thought for 4 seconds >

Lecture Title: Safety Mechanisms for Agentic AI

Audience & Length: Graduate CS/NLP students, 90 minutes

1 Learning Objectives

- Recognize risk vectors unique to autonomous agents.
- Distinguish technical vs. governance safety layers.
- Evaluate and design concrete safeguards for agent deployments.

2 Agenda

Time	Topic	Key Points	Method
------	-------	------------	--------

Thinking

- **Designing a lecture on safety mechanisms for Agentic AI**

The user is asking for a lecture design, which includes an outline, objectives, topics, reading, discussion points, case studies, and evaluation. They're likely aiming for a concise, formal structure—graduate-level, around 90 minutes. No need for a slides deck or document unless they want one later. I'll focus on a structured outline.

- **Designing a concise lecture on Agentic AI safety mechanisms**

The user wants a formal, concise lecture design on safety mechanisms for Agentic AI. I don't need to browse, as the concept is stable and citations are optional. I'll focus on creating a structured, bullet-point outline: introduction, core safety principles, and more.

Gemini: Getting an API key

<https://aistudio.google.com/> (use your PERSONAL email)

1. Make an API key
2. Run `pip install -U google-genai`
3. Download [gemini.py](#) from the class website and make sure you can run it

**** You will need your API key for future activities and HW 2!***

Google AI Studio

<https://aistudio.google.com/>

Google AI Studio

< Dashboard

PROJECT

API Keys

Projects

USAGE & BILLING

Usage

Rate Limit

Spend

Billing

API Keys

API quickstart

Create API key

Group by

API Key

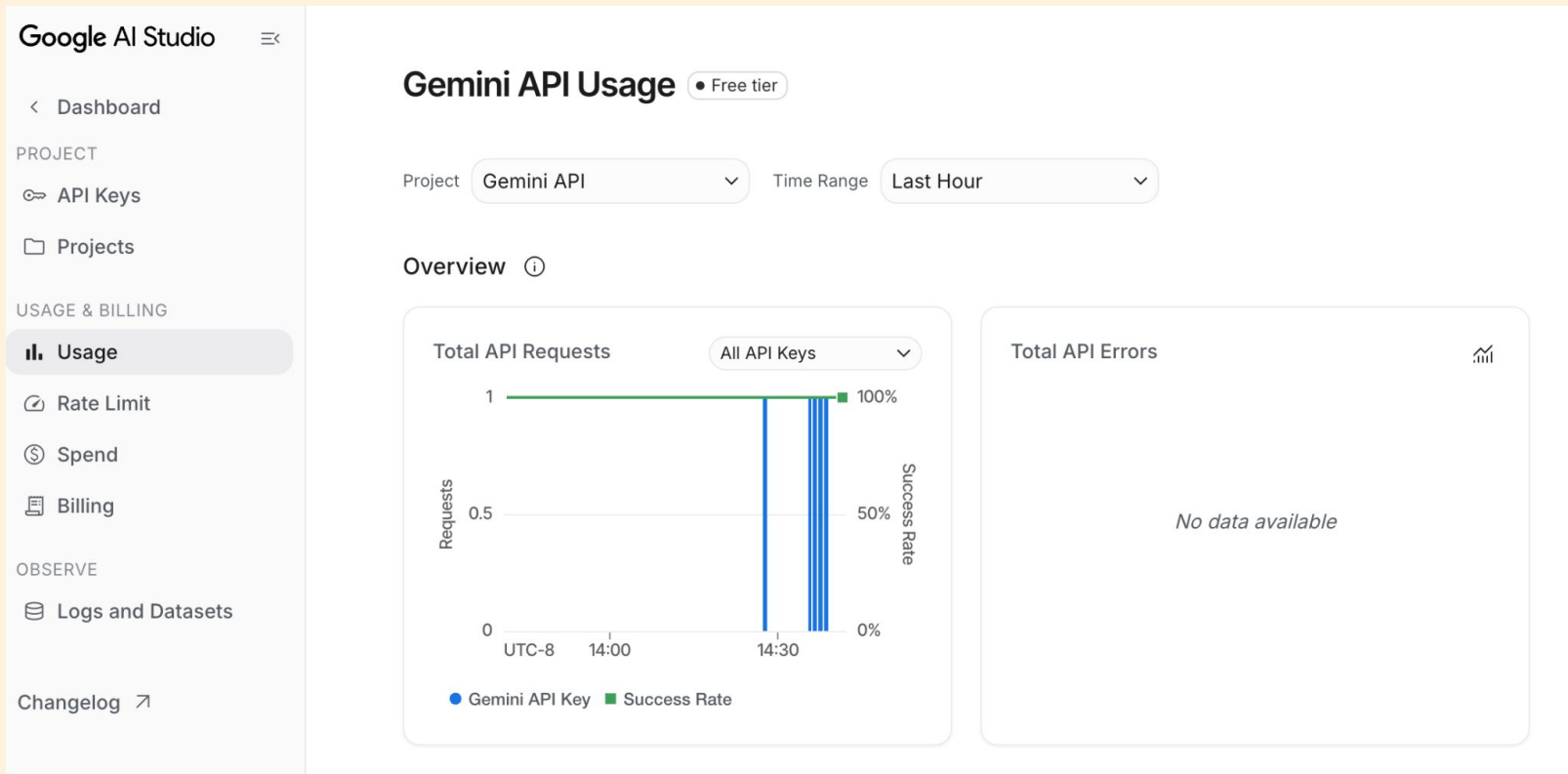
Project

Filter by

All projects

Key	Project	Created	Billing Tier
...gTuw Gemini API Key	Gemini API gen-lang-client-0128119256	Oct 4, 2026	Set up billing Free tier

Google AI Studio <https://aistudio.google.com/>



Google AI Studio <https://aistudio.google.com/>

Google AI Studio

< Dashboard

PROJECT

API Keys

Projects

USAGE & BILLING

Usage

Rate Limit

Spend

Billing

OBSERVE

Logs and Datasets

Changelog

Gemini API Rate Limit

• Free tier

Docs

Set up billing

Project Gemini API

Time Range Last Hour

Rate limits by model

Peak usage per model compared to its limit over the last hour

All models

Compare tiers

Model	Category	RPM ↓	TPM	RPD
Gemini 2.5 Flash	Text-out models	1 / 5	11 / 250K	3 / 20
Gemini 3.5 Flash	Text-out models	1 / 5	11 / 250K	1 / 20
Gemini 3.5 Flash Lite	Text-out models	1 / 15	11 / 250K	1 / 500
Antigravity	Agents	0 / 60	0 / 100K	0 / 100
Deep Research Pro Preview	Agents	0 / 0	0 / 0	0 / 0

Google AI Studio <https://aistudio.google.com/>

Google AI Studio

< Dashboard

PROJECT

API Keys

Projects

USAGE & BILLING

Usage

Rate Limit

Spend

Billing

OBSERVE

Logs and Datasets

Changelog

Gemini API Rate Limit

Free tier

Docs

Set up billing

Project Gemini API

Time Range Last Hour

Rate limits by model

All models

Compare tiers

Peak usage per model compared to its limit over the last hour

Model	Category	RPM ↓	TPM	RPD
Gemini 2.5 Flash	Text-out models	1 / 5	11 / 250K	3 / 20
Gemini 3.5 Flash			1 / 250K	1 / 20
Gemini 3.5 Flash Lite			1 / 250K	1 / 500
Antigravity			/ 100K	0 / 100
Deep Research Pro Preview			/ 0	0 / 0

Please start the HWs early!!
You get 500 free queries per day
with gemini-3.5-flash-lite

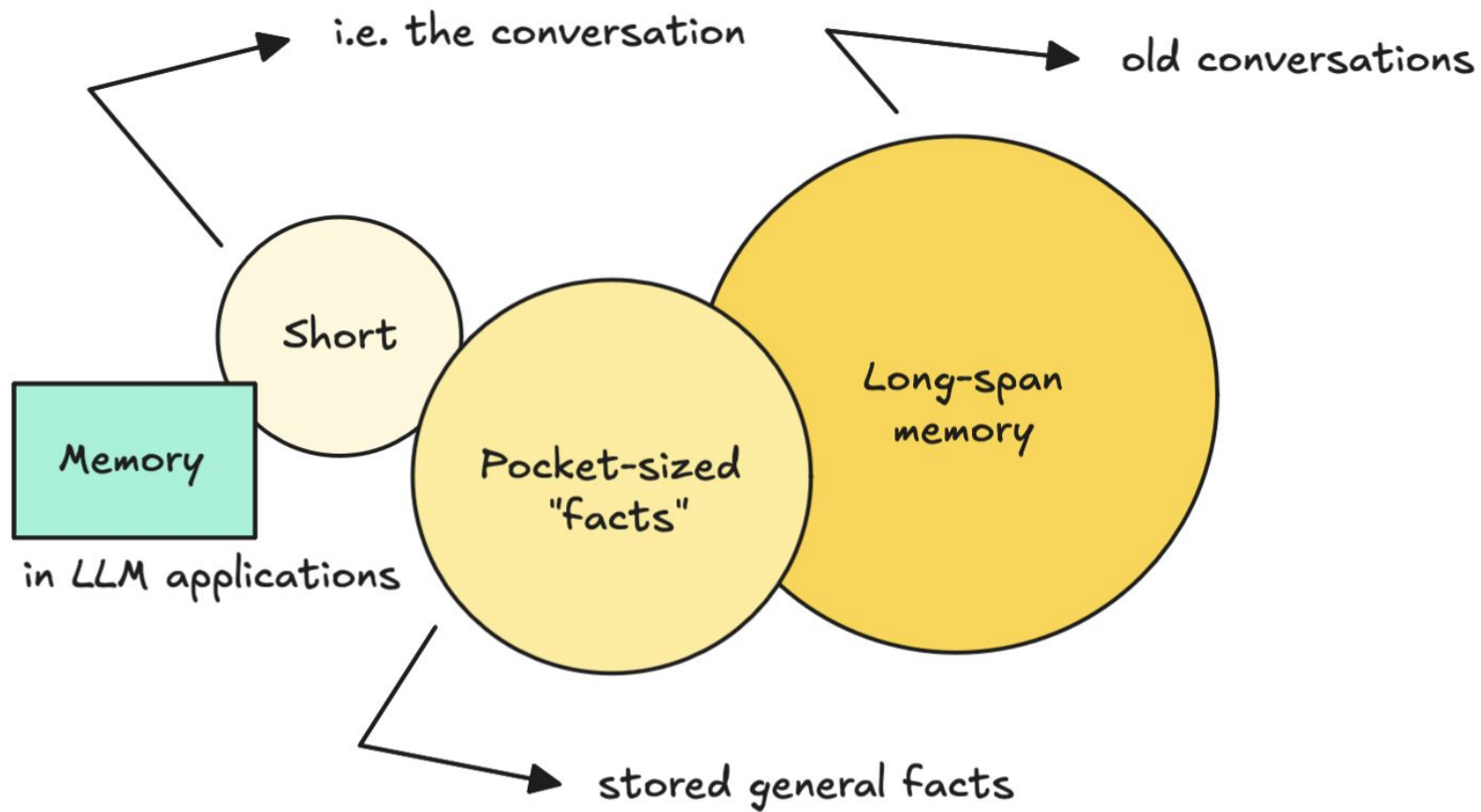
Today's Lecture

1. Prompting
2. Reasoning techniques
3. **Memory management**

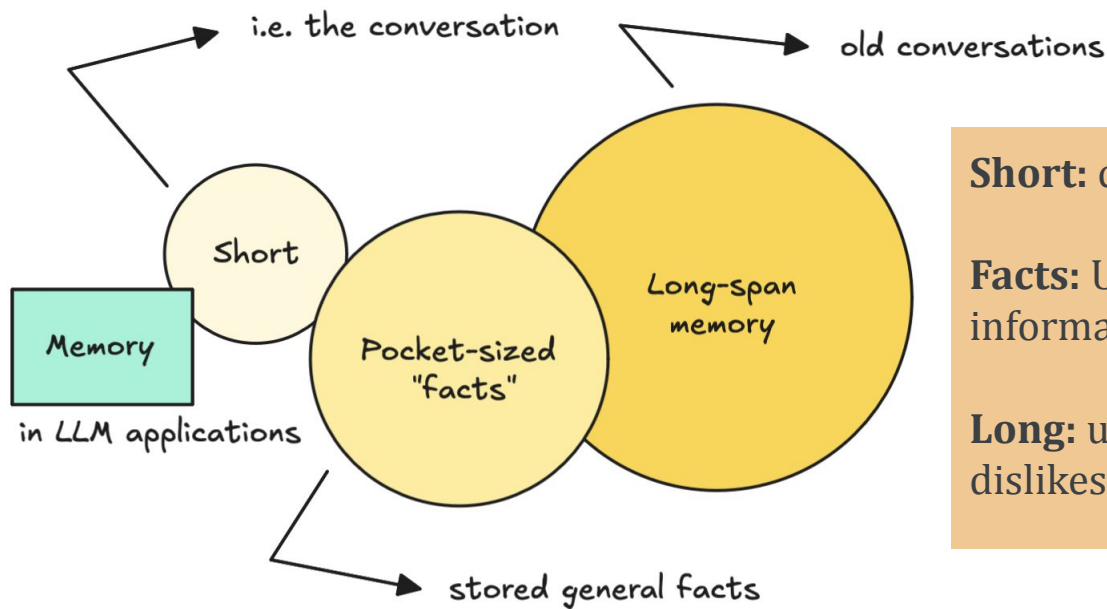
Memory

“Memory” for an LLM means context selection and presentation.

- Your LLM should “remember” pieces of information from prior queries
- You must strategically manage relevant information from prior exchanges to maintain coherence.



Example: a travel agent



Short: current conversation with user

Facts: USD exchange rates, weather information, flight prices/times, etc.

Long: user's dietary restrictions, likes + dislikes, preferred generation style, etc.

Sequential chain

In sequential memory chaining, you append everything.

Pros: Easy to implement, you append each new turn and include the entire conversation chain as the input

Cons: Quickly runs into context limits. Only useful for very short sessions

Sliding window

In sliding window memory, you append only the N last turns.

Pros: Also easy to implement, you keep a fixed-size recent window and drop older turns.

Cons: Does not run into context limits, but can drop important information from earlier turns

Summary-based memory

In summary-based memory, you periodically summarize the conversation

Pros: You can maintain longer-term context under token limits

Cons: Requires iteration to determine how to best summarize information for your specific application

BEFORE (~78% full)



compact →

AFTER (compacted)



Summary-based compaction replaces older conversation turns with an LLM-generated summary to stay within context window limits + shrink input length for a conversation

What is kept during compaction (files, tool outputs, user inputs, etc.) varies depending on task.

Agentic memory in practice

Ways to store information an agent must remember

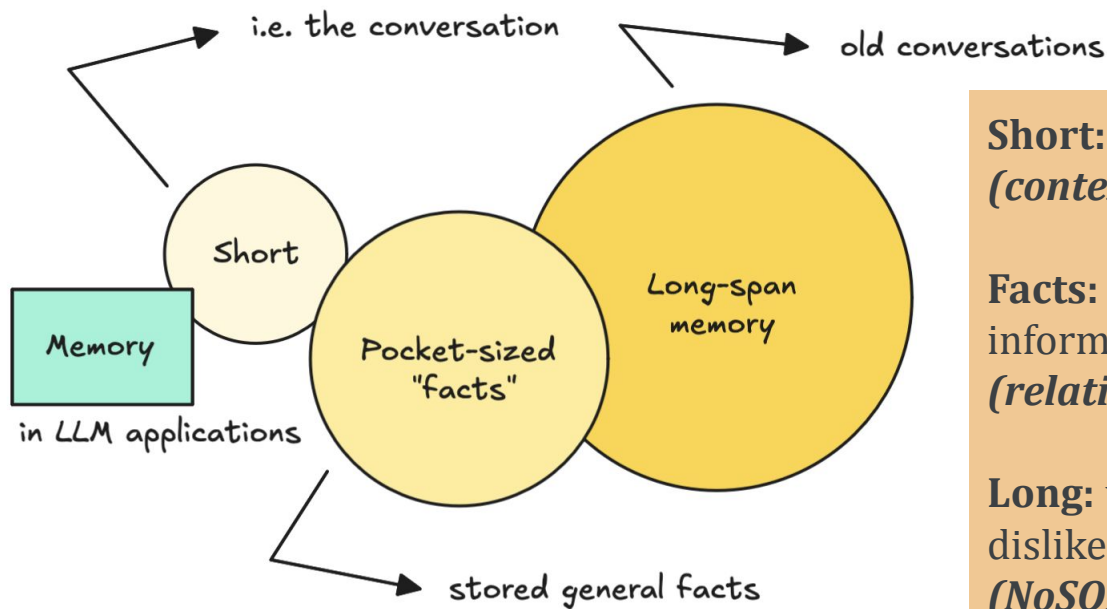
1. Context window
2. In-Memory Cache
3. Relational Database
4. NoSQL Database
5. Vector Database + Embeddings

Agentic memory in practice

Ways to store information an agent must remember

1. Context window: LLM must process immediate, active conversation history
2. In-Memory Cache: Sub-millisecond retrieval of active session states
3. Relational Database: Store structured data (e.g. chat logs, user accounts, billing records)
4. NoSQL Database: Store highly dynamic or unstructured data (e.g. agent trace logs)
5. Vector Database + Embeddings: Perform semantic searches to pull contextually relevant facts based on meaning, not exact keywords (RAG)

Example: a travel agent



Short: current conversation with user
(*context window*)

Facts: USD exchange rates, weather information, flight prices/times, etc.
(*relational database*)

Long: user's dietary restrictions, likes + dislikes, preferred generation style, etc.
(*NoSQL database*)

Discussion (10 min)

Step 1: Think of an LLM task that requires multiple turns, has evolving constraints, depends on remembering prior decisions, and fails if earlier information is forgotten

Step 2: Design a memory store that includes (1) a method to extract relevant information from a conversation, and (2) a storage schema for this information ***under 500 total tokens***